



第10章

算法优化策略



算法设计策略的比较与选择



最大子段和问题

给定由 n 个整数(可能为负整数)组成的序列 $a_1, a_2, \dots, a_n$, 求该序列形如 $\sum_{k=i}^j a_k$ 的子段和的最大值。当所有整数均为负整数时定义其最大子段和为 0。依此定义, 所求的最优值为:

$$\max \left\{ 0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k \right\}$$

例如:

$$A = (-2, 11, -4, 13, -5, -2)$$

$$\text{最大子段和为 } \sum_{k=2}^4 a_k = 20$$



简单算法

```
public static int maxSum()  
{  
    int n=a.length-1;  
    int sum=0;  
    for (int i=1;i<=n;i++) {  
        int thissum=0;  
        for (int j=i;j<=n;j++) {  
            thissum+=a[j];  
            if (thissum>sum) {  
                sum=thissum;  
                besti=i;  
                bestj=j;  
            }  
        }  
    }  
    return sum;  
}
```

注意到 $\sum_{k=i}^j a_k = a_j + \sum_{k=i}^{j-1} a_k$ ，则可将算法中的最后一个for循环省去，避免重复计算只需要 $O(n^2)$ 的计算时间。



分治算法

如果将所给的序列 $a[1:n]$ 分为长度相等的2段 $a[1:n/2]$ 和 $a[n/2+1:n]$, 分别求出这2段的最大子段和,

则 $a[1:n]$ 的最大子段和为

复杂度分析

$$T(n) = \begin{cases} O(1) & n \leq c \\ 2T(n/2) + O(n) & n > c \end{cases}$$

$$T(n) = O(n \log n)$$

(3) $a[1:n]$ 的最大子段和为 $\sum_{k=i}^j a[k]$, 且 $1 \leq i \leq n/2$, $n/2+1 \leq j \leq n$ 。

对定理 4.1(主定理) 设 $a \geq 1$ 和 $b > 1$ 为常数, 设 $f(n)$ 为一函数, $T(n)$ 由递归式

$$T(n) = aT(n/b) + f(n)$$

列对非负整数定义, 其中 n/b 指 $\lfloor n/b \rfloor$ 或 $\lceil n/b \rceil$ 。那么 $T(n)$ 可能有如下的渐近界:

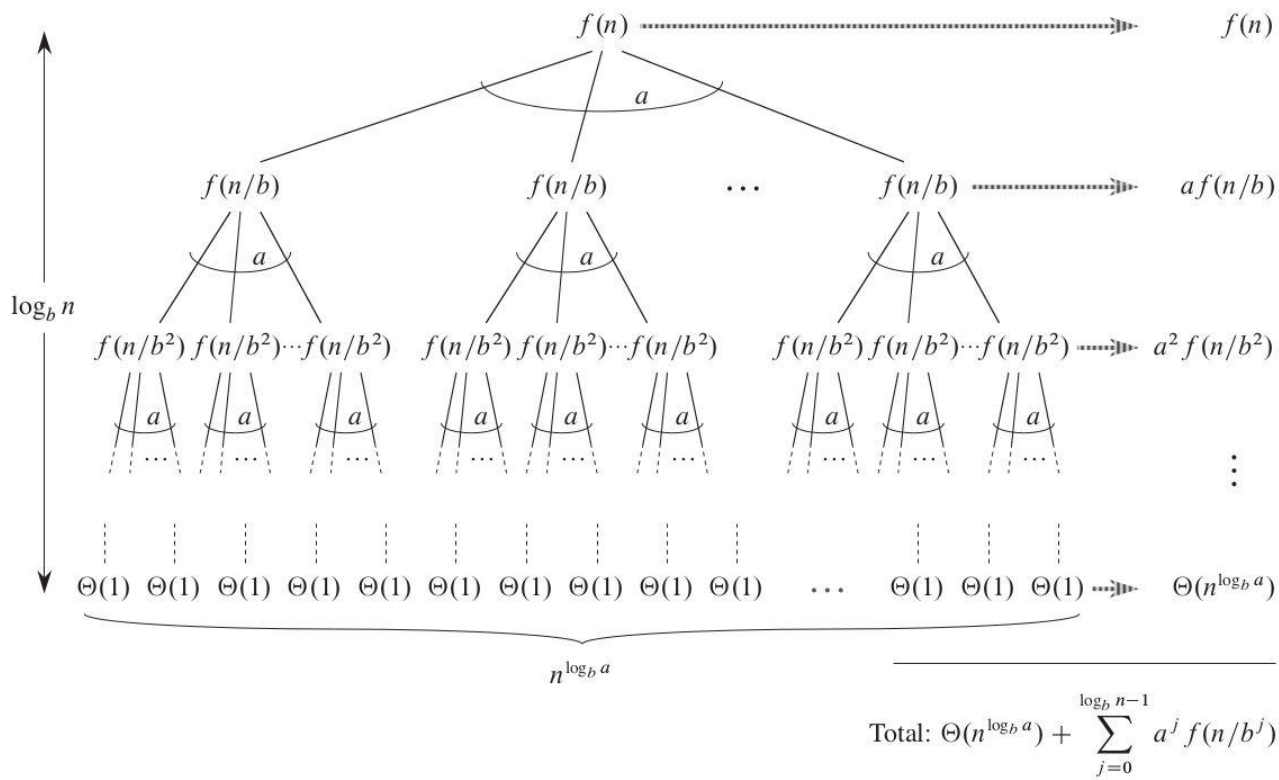
- 1) 若对于某常数 $\epsilon > 0$, 有 $f(n) = O(n^{\log_b a - \epsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$;
- 2) 若 $f(n) = \Theta(n^{\log_b a})$, 则 $T(n) = \Theta(n^{\log_b a} \lg n)$;
- 3) 若对某常数 $\epsilon > 0$, 有 $f(n) = \Omega(n^{\log_b a + \epsilon})$, 且对常数 $c < 1$ 与所有足够大的 n , 有 $af(n/b) \leq cf(n)$, 则 $T(n) = \Theta(f(n))$ 。

治算法。



分治递归式

$$T(n) = aT(n/b) + f(n)$$





主定理

定理 4.1 (主定理) 设 $a \geq 1$ 和 $b > 1$ 为常数, 设 $f(n)$ 为一函数, $T(n)$ 由递归式

$$T(n) = aT(n/b) + f(n)$$

对非负整数定义, 其中 n/b 指 $\lfloor n/b \rfloor$ 或 $\lceil n/b \rceil$ 。那么 $T(n)$ 可能有如下的渐近界:

1) 若对于某常数 $\epsilon > 0$, 有 $f(n) = O(n^{\log_b a - \epsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$;

2) 若 $f(n) = \Theta(n^{\log_b a})$, 则 $T(n) = \Theta(n^{\log_b a} \lg n)$;

3) 若对某常数 $\epsilon > 0$, 有 $f(n) = \Omega(n^{\log_b a + \epsilon})$, 且对常数 $c < 1$ 与所有足够大的 n , 有 $af(n/b) \leq cf(n)$, 则 $T(n) = \Theta(f(n))$ 。 ■

$$(2) f(n) = \Theta(n^{\log_b a})$$

$$\text{设 } g(n) = \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

$$f\left(\frac{n}{b^j}\right) = \Theta\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right)$$

$$\text{则 } \exists c_1, c_2, n_0, \forall n \geq n_0, c_1 \left(\frac{n}{b^j}\right)^{\log_b a} \leq f\left(\frac{n}{b^j}\right) \leq c_2 \left(\frac{n}{b^j}\right)^{\log_b a} \Rightarrow a^j c_1 \left(\frac{n}{b^j}\right)^{\log_b a} \leq a^j f\left(\frac{n}{b^j}\right) \leq a^j c_2 \left(\frac{n}{b^j}\right)^{\log_b a}.$$

$$c_1 n^{\log_b a} \left(\frac{a}{b^{\log_b a}}\right)^j \leq a^j f\left(\frac{n}{b^j}\right) \leq c_2 n^{\log_b a} \left(\frac{a}{b^{\log_b a}}\right)^j \Rightarrow c_1 n^{\log_b a} (1)^j \leq a^j f\left(\frac{n}{b^j}\right) \leq c_2 n^{\log_b a} (1)^j$$

$$c_1 n^{\log_b a} \log_b n \leq g(n) \leq c_2 n^{\log_b a} \log_b n \Rightarrow c_1 n^{\log_b a} \lg n \leq g(n) \leq c_2 n^{\log_b a} \lg n$$

$$\text{所以 } g(n) = \Theta(n^{\log_b a} \lg n)$$

$$T(n) = \Theta(n^{\log_b a}) + \Theta(n^{\log_b a} \lg n) = \Theta(n^{\log_b a} \lg n)$$



$$(1) f(n) = O(n^{(\log_b a) - \varepsilon})$$

$$\text{设 } g(n) = \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

$$f\left(\frac{n}{b^j}\right) = O\left(\left(\frac{n}{b^j}\right)^{(\log_b a) - \varepsilon}\right)$$

$$\text{则 } \exists c, n_0, \forall n \geq n_0, f\left(\frac{n}{b^j}\right) \leq c\left(\frac{n}{b^j}\right)^{(\log_b a) - \varepsilon} \Rightarrow a^j f\left(\frac{n}{b^j}\right) \leq ca^j \left(\frac{n}{b^j}\right)^{(\log_b a) - \varepsilon}.$$

$$a^j f\left(\frac{n}{b^j}\right) \leq cn^{(\log_b a) - \varepsilon} \left(\frac{a}{b^{(\log_b a) - \varepsilon}}\right)^j \Rightarrow a^j f\left(\frac{n}{b^j}\right) \leq cn^{(\log_b a) - \varepsilon} \left(\frac{ab^\varepsilon}{b^{(\log_b a)}}\right)^j \Rightarrow a^j f\left(\frac{n}{b^j}\right) \leq cn^{(\log_b a) - \varepsilon} (b^\varepsilon)^j$$

$$g(n) \leq cn^{(\log_b a) - \varepsilon} \sum_{j=0}^{\log_b n - 1} (b^\varepsilon)^j \Rightarrow g(n) \leq cn^{(\log_b a) - \varepsilon} \frac{1(1 - (b^\varepsilon)^{\log_b n})}{1 - b^\varepsilon} \Rightarrow g(n) \leq cn^{(\log_b a) - \varepsilon} \frac{(1 - n^\varepsilon)}{1 - b^\varepsilon}$$

$$\Rightarrow g(n) \leq c \frac{(n^{(\log_b a) - \varepsilon} - n^{\log_b a})}{1 - b^\varepsilon}$$

$$\text{所以 } g(n) = O(n^{\log_b a})$$

$$T(n) = \Theta(n^{\log_b a}) + O(n^{\log_b a}) = \Theta(n^{\log_b a})$$



(3) $f(n) = \Omega(n^{(\log_b a) + \varepsilon})$ 存在 $c < 1$, 使得 $af(\frac{n}{b}) \leq cf(n)$

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(\frac{n}{b^j})$$

$$af(\frac{n}{b}) \leq cf(n) \Rightarrow a^j f(\frac{n}{b^j}) \leq c^j f(n) \Rightarrow g(n) \leq \sum_{j=0}^{\log_b n - 1} c^j f(n)$$

$$\Rightarrow g(n) \leq \sum_{j=0}^{\infty} c^j f(n) \Rightarrow g(n) \leq \frac{1}{1-c} f(n)$$

因此 $g(n) = O(f(n))$

$$\text{又因为 } g(n) = f(n) + af(\frac{n}{b}) + a^2 f(\frac{n}{b^2}) + \dots + a^{\log_b n - 1} f(\frac{n}{b^{\log_b n - 1}}) \geq f(n)$$

因此 $g(n) = \Omega(f(n))$

所以 $g(n) = \Theta(f(n))$

$T(n) = \Theta(n^{\log_b a}) + \Theta(f(n))$, 又因为 $f(n) = \Omega(n^{(\log_b a) + \varepsilon})$, 所以 $T(n) = \Theta(f(n))$



动态规划算法

记 $b[j] = \max_{1 \leq i \leq j} \{ \sum_{k=i}^j a[k] \}$, $1 \leq j \leq n$, 则所求的最大子段和为

$$\max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a[k] = \max_{1 \leq j \leq n} \max_{1 \leq i \leq j} \sum_{k=i}^j a[k] = \max_{1 \leq j \leq n} b[j]$$

当 $b[j-1] > 0$ 时 $b[j] = b[j-1] + a[j]$, 否则 $b[j] = a[j]$ 。由此可得计算 $b[j]$ 的动态规划递归式: $b[j] = \max\{b[j-1] + a[j], a[j]\}$, $1 \leq j \leq n$

```
public static int maxSum()
{
    int n=a.length-1;
    int sum=0,
        b=0;
    for (int i=1;i<=n;i++) {
        if (b>0) b+=a[i];
        else b=a[i];
        if (b>sum)sum=b;
    }
    return sum;
}
```

算法显然需要 $O(n)$ 计算时间和 $O(n)$ 空间。



最大子矩阵和问题

给定一个 m 行 n 列的整数矩阵 a ，试求矩阵 a 的一个子矩阵，使其各元素之和为最大。

记
$$s(i1, i2, j1, j2) = \sum_{i=i1}^{i2} \sum_{j=j1}^{j2} a[i][j]$$

最大子矩阵和问题的最优值为
$$\max_{\substack{1 \leq i1 \leq i2 \leq m \\ 1 \leq j1 \leq j2 \leq n}} s(i1, i2, j1, j2)$$

由于
$$\max_{\substack{1 \leq i1 \leq i2 \leq m \\ 1 \leq j1 \leq j2 \leq n}} s(i1, i2, j1, j2) = \max_{1 \leq i1 \leq i2 \leq m} \{ \max_{1 \leq j1 \leq j2 \leq n} s(i1, i2, j1, j2) \} = \max_{1 \leq i1 \leq i2 \leq m} t(i1, i2)$$

其中,
$$t(i1, i2) = \max_{1 \leq j1 \leq j2 \leq n} s(i1, i2, j1, j2) = \max_{1 \leq j1 \leq j2 \leq n} \sum_{j=j1}^{j2} \sum_{i=i1}^{i2} a[i][j]$$

设 $b[j] = \sum_{i=i1}^{i2} a[i][j]$ ，则
$$t(i1, i2) = \max_{1 \leq j1 \leq j2 \leq n} \sum_{j=j1}^{j2} b[j]$$

由于解最大子段和问题的动态规划算法需要时间 $O(n)$ ，故算法的双重for循环需要计算时间 $O(m^2n)$ 。



最大 m 子段和问题

给定由 n 个整数(可能为负整数)组成的序列 $a_1, a_2, \dots, a_n$, 以及一个正整数 m , 要求确定序列的 m 个不相交子段, 使这 m 个子段的总和达到最大。

设 $b(i, j)$ 表示数组 a 的前 j 项中 i 个子段和的最大值, 且第 i 个子段含 $a[j]$ ($1 \leq i \leq m, 1 \leq j \leq n$)。则所求的最优值显然为 $\max_{m \leq j \leq n} b(m, j)$ 与最大子段和问题类似地, 计算 $b(i, j)$ 的递归式为

$$b(i, j) = \max \{b(i, j-1) + a[j], \max_{i-1 \leq t < j} b(i-1, t) + a[j]\} \quad (1 \leq i \leq m, 1 \leq j \leq n)$$

初始时, $b(0, j)=0, (1 \leq j \leq n); b(i, 0)=0, (1 \leq i \leq m)$ 。

优化: 注意到在上述算法中, 计算 $b[i][j]$ 时只用到数组 b 的第 $i-1$ 行和第 i 行的值。因而算法中只要存储数组 b 的当前行, 不必存储整个数组。另一方面, $b(i-1, t)$ 的值可以在计算第 $i-1$ 行时预先计算并保存起来。计算第 i 行的值时不必重新计算, 节省了计算时间和空间。



最长单调递增子序列 (LIS)

问题：在数组 $a[0..n-1]$ 中存放有 n 个数，找出其中最长的单调递增子序列，如 $a[] = \{10, 4, 8, 5, 3, 6, 9, 7, 12\}$ ，其最长单调递增子序列为 $\{4, 5, 6, 7, 12\}$ 。

问题分析：用数组 $L[0..n-1]$ 记录以 $a[i]$ 为结尾元素的最长单调递增子序列的长度，则数组 a 的最长单调递增子序列的长度为 $\max\{L[i] | 0 \leq i < n\}$ 。

可以递归地定义：

对于 $j < i < n$ 且 $a[i] > a[j]$ ，有 $L[i] = \max\{L[j] + 1, L[i]\}$

容易证明， $L[i]$ 满足最优子结构性质。



最长单调递增子序列

- 假设数组 $A=\{3, 5, 7, 1, 2, 8\}$
 - 初始化 $n=6$ 的数组 L , $L=\{1,1,1,1,1,1\}$
 - 遍历 i , 计算 $L[i]$
 - 对于每个 i , 遍历 $j(0<j<i)$
 - 如果 $a[i]>a[j]$, 则 $L[i]=\max\{L[j]+1, L[i]\}$

Diagram illustrating the calculation of the Longest Increasing Subsequence (LIS) for the array $A = \{3, 5, 7, 1, 2, 8\}$. The table shows the values of $L[i]$ for each i , with the maximum value 4 highlighted in red at $i=8, j=3$.

		3	5	7	1	2	8
3	1						
5	2	1					
7	2	3	1				
1	1	1	1	1			
2	1	1	1	2	1		
8	2	3	4	2	3	1	

最大值



最长单调递增子序列

设计算法如下：

```
def get_lis_length(a)
{
    lis = np.ones(n)
    for (i=1; i<n; i++) {
        for (j=0; j<i; j++)
        {
            if (a[i]>a[j]) lis[i]=max(lis[j]+1, lis[i]);
        }
    }
    return int(lis.max());
}
```

分析算法可知，算法的时间复杂度为 $O(n^2)$ 。



最长单调递增子序列

二分查找：

原始数组为A，建立一个辅助数组B，变量end记录B数组末尾元素的下标

遍历A中的所有的元素 $x=A[i]$

如果 $x > B$ 的末尾元素，则将x追加到B的末尾， $end += 1$

如果 $x < B$ 的末尾元素，则利用二分法查找，寻找B中第一个大于x的元素，并用x进行替换，例如 $x=4$ ， $B=[1,3,5,6]$ 变为 $[1,3,4,6]$

遍历结束之后，B的长度则为LIS的长度



最长单调递增子序列

INPUT	A : [2, 1, 5, 3, 6, 4, 8, 9, 7]	
初始化	B[0] = A[0] end=0	B : [2]
迭代		
1	将A[1] 放到B里面 A : [2, 1 , 5, 3, 6, 4, 8, 9, 7]	因为 A[1] <= B[0], 替换 B : [1]
2	将A[2] 放到B里面 A : [2, 1, 5 , 3, 6, 4, 8, 9, 7]	因为 A[2] > B[0], B追加, 修改end B : [1, 5] end = 1
3	将A[3] 放到B里面 A : [2, 1, 5, 3 , 6, 4, 8, 9, 7]	因为 A[3] <= B[1], 替换 B : [1, 3]
4	将A[4] 放到B里面 A : [2, 1, 5, 3, 6 , 4, 8, 9, 7]	因为 A[4] > B[1], B追加, 修改end B : [1, 3, 6] end=2
5	将A[5] 放到B里面 A : [2, 1, 5, 3, 6, 4 , 8, 9, 7]	因为 A[5] <= B[2], 替换 B : [1, 3, 4] end=2
6	将A[6] 放到B里面 A : [2, 1, 5, 3, 6, 4, 8 , 9, 7]	因为 A[6] > B[2], B追加, 修改end B : [1, 3, 4, 8] end=3
7	将A[7] 放到B里面 A : [2, 1, 5, 3, 6, 4, 8, 9 , 7]	因为 A[7] > B[3], B追加, 修改end B : [1, 3, 4, 8, 9] end=4
8	将A[8] 放到B里面 A : [2, 1, 5, 3, 6, 4, 8, 9, 7]	因为 A[8] <= B[4], 替换 B : [1, 3, 4, 7, 9]
OUTPUT	end + 1 = 5	知乎 @企鵝



最长单调递增子序列

```
def get_lis_length(arr):  
    """  
    二分法获取最长递增子序列长度  
    :param arr:  
    :return:  
    """  
    temp = [arr[0]]  
    end = 0  
  
    for i in range(1, len(arr)):  
        if arr[i] > temp[end]:  
            end += 1  
            temp.append(arr[i])  
        else :  
            pos = binary_search(temp, 0, len(temp), arr[i])  
            temp[pos] = arr[i]  
    return end + 1
```

```
def binary_search(arr, start, end, value):  
    l = start  
    r = end - 1  
    while l <= r:  
        m = (l + r) // 2  
        if arr[m] == value:  
            return m  
        elif arr[m] < value:  
            l = m + 1  
        else:  
            r = m - 1  
    return l
```



动态规划加速原理



货物储运问题

在一个铁路沿线顺序存放着 n 堆装满货物的集装箱。货物储运公司要将集装箱有次序地集中成一堆。规定每次只能选相邻的2堆集装箱合并成新的一堆，所需的运输费用与新的一堆中集装箱数成正比。给定各堆的集装箱数，试制定一个运输方案，使总运输费用最少。

设合并 $a[i:j]$ ， $1 \leq i \leq j \leq n$ ，所需的最少费用为 $m[i,j]$ ，则原问题的最优值为 $m[1, n]$ 。由最优子结构性质可知，

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i < k \leq j} \{m[i, k-1] + m[k, j] + \sum_{t=i}^j a[t]\} & i < j \end{cases}$$

根据递归式，按通常方法可设计计算 $m(i,j)$ 的 $O(n^3)$ 动态规划算法



四边形不等式

货物储运问题的动态规划递归式是下面更一般的递归计算式的特殊情形。

$$m[i, j] = \begin{cases} 0 & i = j \\ w(i, j) + \min_{i < k \leq j} \{m[i, k-1] + m[k, j]\} & i < j \end{cases}$$

对于 $i \leq i' \leq j \leq j'$, 当函数 $w(i, j)$ 满足

$w(i, j) + w(i', j') \leq w(i', j) + w(i, j')$
时称 w 满足 **四边形不等式**。

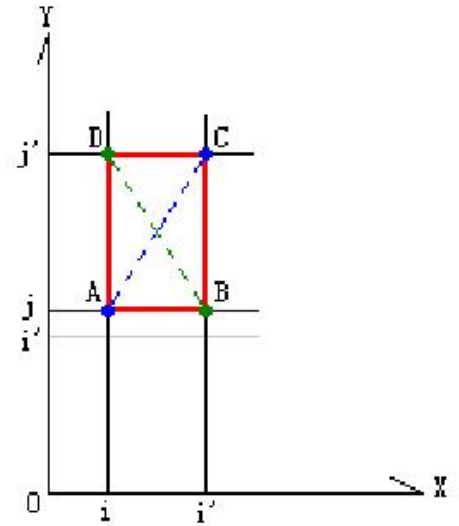
当函数 $w(i, j)$ 满足

$$w(i', j) \leq w(i, j')$$

时称 w **关于区间包含关系单调**

对于满足四边形不等式的单调函数 w , 可推知由递归式定义的函数 $m(i, j)$ 也满足四边形不等式, 即

$$m(i, j) + m(i', j') \leq m(i', j) + m(i, j')$$





四边形不等式

- 定义 $m_k(i,j)=m(i,k)+m(k+1,j)+w(i,j)$ 表示分割为 k 时的 $m(i,j)$ 值, 对于 $i \leq i' \leq j \leq j'$ 时, 记

$u = \operatorname{argmin}_{i \leq k < j'} m_k(i, j')$, $v = \operatorname{argmin}_{i' \leq k < j} m_k(i', j)$ 分

别表示 $m(i, j')$ 和 $m(i', j)$ 的最优分割点, 对四边形的长度 $j'-i$ 运动归纳法, 当 $i=i'$ 或是 $j=j'$ 时, m 的四边形不等式显成立, 于是长度 ≤ 1 时, m 的四边形不等式成立

先考虑 $i'=j$ 的情形



四边形不等式

- 对于 $i \leq i' = j \leq j'$, 若 $u < j$, 则

$$m(i, j) \leq m(i, u) + m(u + 1, j) + w(i, j)$$

由归纳法, 当长度为 $j' - (u + 1)$ 时, 四边形不等式成立, 即

$$m(u + 1, j) + m(i', j') \leq m(u + 1, j') + m(i', j), \text{ 于是}$$

$$m(i, j) + m(u + 1, j) + m(i', j') \leq m(i, u) + m(u + 1, j) + w(i, j) + m(u + 1, j') + m(i', j)$$

$$m(i, j) + m(i', j') \leq m(i, u) + m(u + 1, j') + m(i', j) + w(i, j)$$

对 $w(i, j)$ 运用区间包含单调性 $w(i, j) \leq w(i, j')$, 得到

$$\begin{aligned} m(i, j) + m(i', j') &\leq m(i, u) + m(u + 1, j') + w(i, j') + m(i', j) \\ &= m(i, j') + m(i', j) \end{aligned}$$



四边形不等式

■ 若 $u \geq j$, 则

$$m(i', j') \leq m(i', u) + m(u + 1, j') + w(i', j')$$

由归纳法, 当长度为 $u - i$ 时, 四边形不等式成立, 即

$$m(i, j) + m(i', u) \leq m(i, u) + m(i', j), \text{ 于是}$$

$$m(i, j) + m(i', u) + m(i', j') \leq m(i', u) + m(u + 1, j') + w(i', j') + m(i, u) + m(i', j)$$

$$m(i, j) + m(i', j') \leq m(i, u) + m(u + 1, j') + w(i', j') + m(i', j)$$

对 $w(i', j')$ 运用区间包含单调性 $w(i', j') \leq w(i, j')$, 得到

$$\begin{aligned} m(i, j) + m(i', j') &\leq m(i, u) + m(u + 1, j') + w(i, j') + m(i', j) \\ &= m(i, j') + m(i', j) \end{aligned}$$



四边形不等式

- 对于 $i \leq i' < j \leq j'$ (不需要区间包含单调性), 若 $u \leq v$, 则 $i' \leq u \leq v < j, i' \leq v < j \leq j'$, 即 $i \leq u < j, i' \leq v < j'$, 因此

$$m(i, j) \leq m(i, u) + m(u + 1, j) + w(i, j)$$

$$m(i', j') \leq m(i', v) + m(v + 1, j') + w(i', j')$$

由 $u + 1 \leq v + 1 \leq j \leq j'$, 及归纳法, 当长度为 $j' - (u + 1)$ 时,

四边形不等式成立, 即

$m(u + 1, j) + m(v + 1, j') \leq m(u + 1, j') + m(v + 1, j)$, 于是

$$\begin{aligned} & m(i, j) + m(i', j') + m(u + 1, j) + m(v + 1, j') \\ & \leq m(i, u) + m(u + 1, j) + w(i, j) \\ & \quad + m(i', v) + m(v + 1, j') + w(i', j') \\ & \quad + m(u + 1, j') + m(v + 1, j) \end{aligned}$$



四边形不等式

消去 $m(u+1, j) + m(v+1, j')$, 得到

$$\begin{aligned} & m(i, j) + m(i', j') \\ & \leq m(i, u) + m(u+1, j') + w(i, j) \\ & \quad + m(i', v) + m(v+1, j) + w(i', j') \end{aligned}$$

对 $w(i, j)$ 运用四边形不等式 $w(i, j) + w(i', j') \leq w(i, j') + w(i', j)$, 得到

$$\begin{aligned} & m(i, j) + m(i', j') \\ & \leq m(i, u) + m(u+1, j') + w(i, j') \\ & \quad + m(i', v) + m(v+1, j) + w(i', j) \\ & = m(i, j') + m(i', j) \end{aligned}$$

■ 若 $u > v$, 则 $i \leq v < u < j, i' \leq (v < u) \leq j'$, 因此

$$m(i, j) \leq m(i, v) + m(v+1, j) + w(i, j)$$

$$m(i', j') \leq m(i', u) + m(u+1, j') + w(i', j')$$



四边形不等式

由 $i \leq i' \leq v < u$, 及归纳法, 当长度为 $u - i$ 时,

$m(i, v) + m(i', u) \leq m(i, u) + m(i', v)$, 于是

$$\begin{aligned} & m(i, j) + m(i', j') \\ & \leq m(i, v) + m(v + 1, j) + w(i, j) \\ & \quad + m(i', u) + m(u + 1, j') + w(i', j') \\ & \leq m(i, u) + m(u + 1, j') + w(i, j) \\ & \quad + m(i', v) + m(v + 1, j) + w(i', j') \end{aligned}$$

对 $w(i, j)$ 运用四边形不等式 $w(i, j) + w(i', j') \leq w(i, j') + w(i', j)$, 得到

$$\begin{aligned} & m(i, j) + m(i', j') \\ & \leq m(i, u) + m(u + 1, j') + w(i, j') \\ & \quad + m(i', v) + m(v + 1, j) + w(i', j) \\ & = m(i, j') + m(i', j) \end{aligned}$$



四边形不等式

- 若 $m(i,j)$ 满足四边形不等式，记 $s(i,j)=\operatorname{argmin}_{i \leq k < j} m_k(i,j)$ 表示最优分割点，则有
$$s(i,j-1) \leq s(i,j) \leq s(i+1,j)$$
- 记 $u=s(i,j), k_1=s(i,j-1), k_2=s(i+1,j)$
- 若 $k_1 > u$, 则 $u+1 \leq k_1+1 \leq j-1 \leq j$, 根据 $m(i,j)$ 四边形不等式有

$$m(u+1, j-1) + m(k_1+1, j) \leq m(u+1, j) + m(k_1+1, j-1)$$



四边形不等式

跟据 u 是 $m(i, j)$ 的最优分割点, 得到

$$m(i, j) = m(i, u) + m(u + 1, j) \leq m(i, k_1) + m(k_1 + 1, j)$$

将两不等式相加, 得到

$$\begin{aligned} m(u + 1, j - 1) + m(k_1 + 1, j) + m(i, u) + m(u + 1, j) \\ \leq m(u + 1, j) + m(k_1 + 1, j - 1) \\ + m(i, k_1) + m(k_1 + 1, j) \end{aligned}$$

即

$$m(i, u) + m(u + 1, j - 1) \leq m(i, k_1) + m(k_1 + 1, j - 1)$$

与 k_1 是 $m(i, j - 1)$ 的最优分割点矛盾, 因此 $s(i, j - 1) \leq s(i, j)$



四边形不等式

- 若 $u > k_2$, 则 $i \leq i+1 \leq k_2 \leq u$, 由四边形不等式得

$$m(i, k_2) + m(i+1, u) \leq m(i, u) + m(i+1, k_2)$$

跟据 k_2 是 $m(i+1, j)$ 的最优分割点, 得到

$$m(i+1, j) = m(i+1, k_2) + m(k_2+1, j) \leq m(i+1, u) + m(u+1, j)$$

将两不等式相加, 得到

$$\begin{aligned} m(i, k_2) + m(i+1, u) + m(i+1, k_2) + m(k_2+1, j) \\ \leq m(i, u) + m(i+1, k_2) \\ + m(i+1, u) + m(u+1, j) \end{aligned}$$

即

$$m(i, k_2) + m(k_2+1, j) \leq m(i, u) + m(u+1, j)$$

与 u 是 $m(i, j)$ 的最优分割点矛盾, 因此 $s(i, j) \leq s(i+1, j)$



四边形不等式

- 因此在计算状态 $m(i,j)$ 的同时将其最优分割点 $s(i,j)$ 记录下来，则对于最优分割点 k 的枚举量降为

$$\min_{i < k \leq j} \{m(i, k-1) + m(k, j)\} = \min_{s(i, j-1) \leq k \leq s(i+1, j)} \{m(i, k-1) + m(k, j)\}$$

- 改进后动态规划算法求解 $m(1,n)=\text{opt}\{m(i,j)\}$ 所需的时间为

$$\begin{aligned} \sum_{1 \leq i < j \leq n} \{s(i+1, j) - s(i, j-1) + 1\} &= \sum_{j=1}^n \sum_{i=1}^{j-1} \{s(i+1, j) - s(i, j-1) + 1\} \\ &= \sum_{r=1}^{n-1} \sum_{q=1}^{n-r} s(q+1, q+r) - s(q, q+r-1) + 1 = \sum_{r=1}^{n-1} n-r + s(n-r, n) - s(1, r) \end{aligned}$$



货物储运问题

- 对于货物储运问题, $w(i,j)=cnt(j)-cnt(i-1)$, 其中 $cnt(n)=a[1]+\dots+a[n]$
- 对于 $i < j$, 要证明
$$w(i,j)+w(i+1,j+1) \leq w(i+1,j)+w(i,j+1)$$
 - 移项得 $w(i,j)-w(i+1,j) \leq w(i,j+1)-w(i+1,j+1)$
 - 带入 cnt , $w(i,j)-w(i+1,j)=cnt(i)-cnt(i-1) \leq w(i,j+1)-w(i+1,j+1)$
 - 且满足区间包含单调性



问题的算法特征



贪心策略

- 采用每次合并集装箱数最少的相邻2堆货物的贪心策略，并不能得到最优解。

- 例如： 5, 3, 4, 1, 3, 2, 3, 4

5, 3, 4, 4, 2, 3, 4

5, 3, 4, 4, 5, 4

5, 7, 4, 5, 4

5, 7, 9, 4

12, 9, 4

12, 13

25

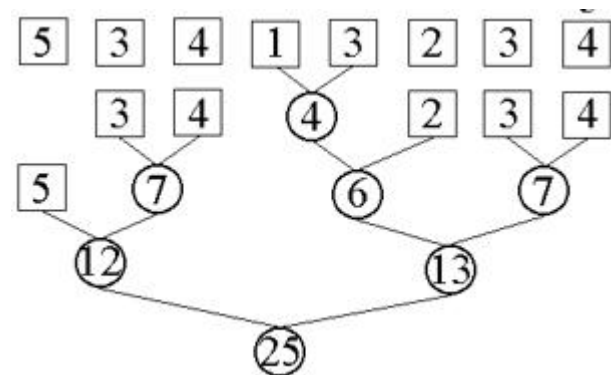
$$4+5+7+9+12+13+25=75$$



贪心策略

5, 3, 4, 1, 3, 2, 3, 4
5, 3, 4, 4, 2, 3, 4
5, 3, 4, 4, 5, 4
5, 7, 4, 5, 4
5, 7, 4, 9
5, 11, 9
16, 9
25

$$4+5+7+9+11+16+25=77$$



$$4+6+7+7+12+13+25=74$$



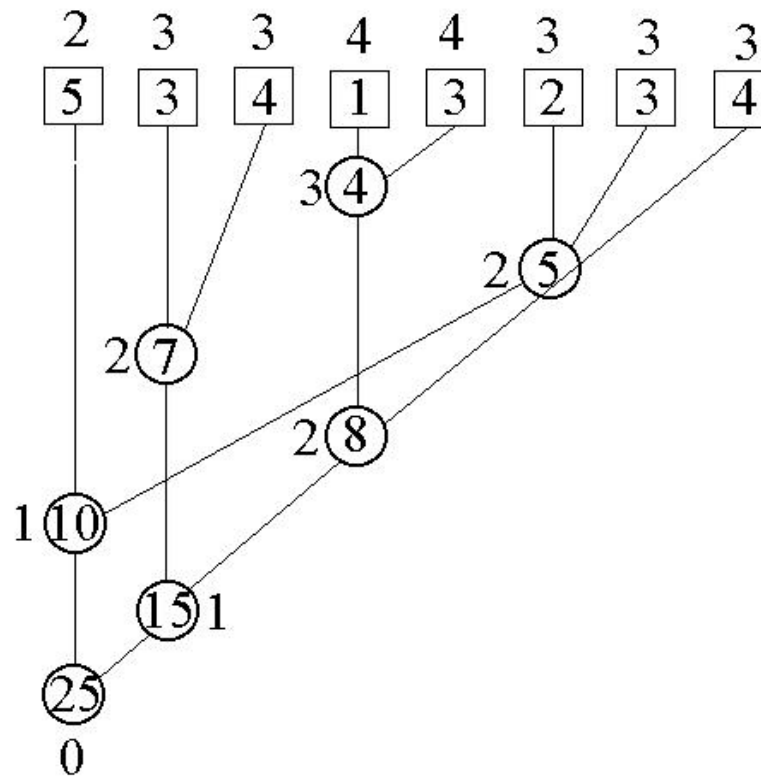
贪心策略

- 适当放松相邻性约束，引入相容结点对概念。如图，原始结点用方形结点表示，合并生成的结点用圆形结点表示。
- 最小相容结点对 $a[i]$ 和 $a[j]$ 是满足下面条件的结点对：
 - (1) 结点 $a[i]$ 和 $a[j]$ 之间没有方形结点；
 - (2) 在所有满足条件 (1) 的结点中 $a[i]+a[j]$ 的值最小；
 - (3) 在所有满足条件 (1) 和 (2) 的结点中下标 i 最小；
 - (4) 在所有满足条件 (1) ~ (3) 的结点中下标 j 最小。



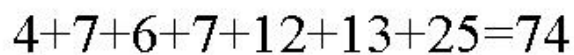
贪心策略

- 相应的最小相容合并树，如图所示。



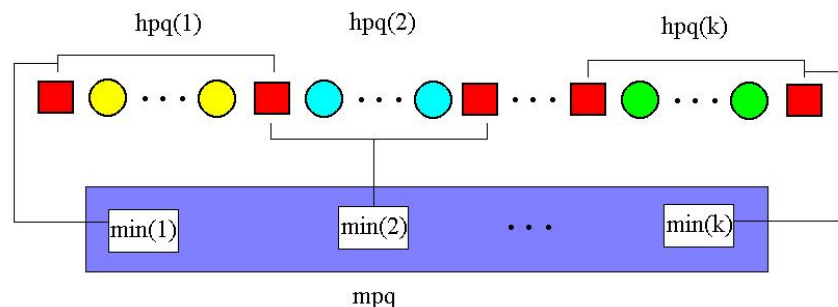


根据上述定理，容易从各原始结点在相容合并树中所处的层序构造出相应的最优合并树，如图所示。





算法



1. **组合阶段:** 将给定的 n 个数作为方形结点依序从左到右排列, $a[1], a[2], \dots, a[n]$ 。反复删除序列中最小相容结点对 $a[i]$ 和 $a[j]$, ($i < j$), 并在位置 i 处插入值为 $a[i] + a[j]$ 的圆形结点, 直至序列中只剩下1个结点。 $O(n \log n)$
2. **标记层序阶段:** 将第一阶段结束后留下的惟一结点标记为第0层结点。然后以与第一阶段相反的组合顺序标记其余结点的层序。 $O(n)$
3. **重组阶段:** 根据标记层序阶段计算出的各结点的层序, 按下述规则重组。 $O(n)$
结点 $a[i]$ 和 $a[j]$ 重组为新结点应满足:
 - (1) $a[i]$ 和 $a[j]$ 在当前序列中相邻;
 - (2) $a[i]$ 和 $a[j]$ 均为当前序列中最大层序结点;
 - (3) 在所有满足条件 (1) 和 (2) 的结点中, 下标 i 最小。

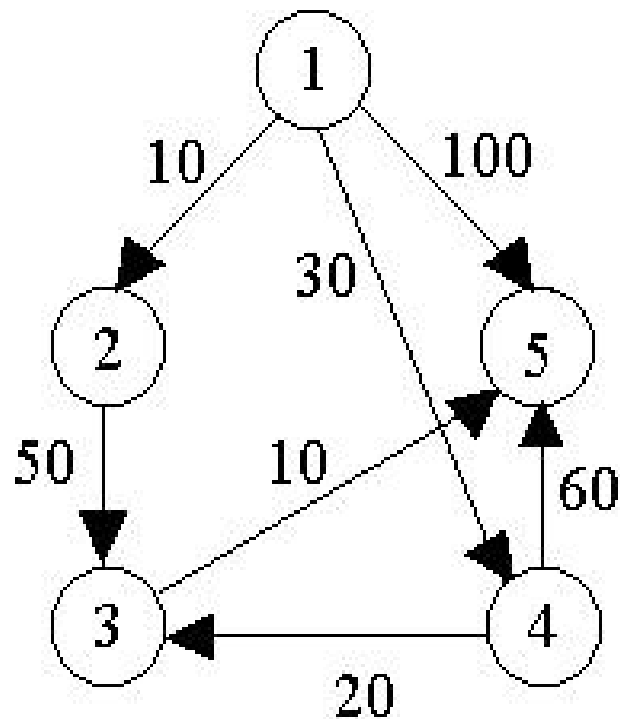


优化数据结构



带权区间最短路问题

例如，对右图中的有向图，应用Dijkstra算法计算从源顶点1到其他顶点间最短路径的过程列在下页的表中。





带权区间最短路问题

Dijkstra算法的迭代过程:

迭代	S	u	dist[2]	dist[3]	dist[4]	dist[5]
初始	{1}	—	10	maxint	30	100
1	{1, 2}	2	10	60	30	100
2	{1, 2, 4}	4	10	50	30	90
3	{1, 2, 4, 3}	3	10	50	30	60
4	{1, 2, 4, 3, 5}	5	10	50	30	60



带权区间最短路问题

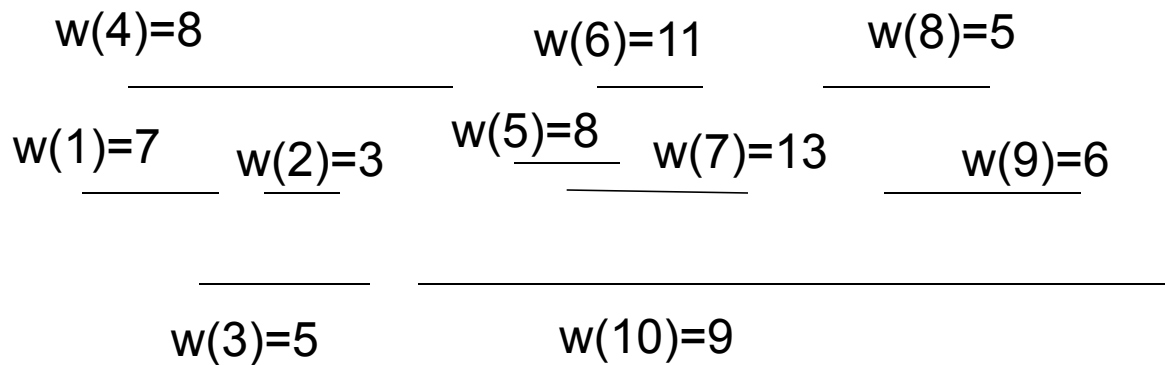
S 是直线上 n 个带权区间的集合。从区间 $I \in S$ 到区间 $J \in S$ 的一条路是 S 的一个区间序列 $J(1), J(2), \dots, J(k)$, 其中 $J(1) = I, J(k) = J$, 且对所有 $1 \leq i \leq k-1$, $J(i)$ 与 $J(i+1)$ 相交。这条路的长度定义为路上各区间权之和。在所有从 I 到 J 的路中, 路长最短的路称为从 I 到 J 的最短路。带权区间图的单源最短路问题要求计算从 S 中一个特定的源区间到 S 中所有其他区间之间的最短路。

区间集 $S(i)$ 的扩展定义为: $S(i) \cup T$, 其中 T 是满足下面条件的另一区间集。 T 中任意区间 $I=[a,b]$ 均有 $b > b(i)$ 。

设区间 $I(k)$ ($k \leq i$) 是区间集 $S(i)$ 中的一个区间, $1 \leq i \leq n$ 。如果对于 $S(i)$ 的任意扩展 $S(i) \cup T$, 当区间 $J \in T$ 且在 $S(i) \cup T$ 中有从 $I(1)$ 到 J 的路时, 在 $S(i) \cup T$ 中从 $I(1)$ 到 J 的任一最短路都不含区间 $I(k)$, 则称区间 $I(k)$ 是 $S(i)$ 中的无效区间。若 $S(i)$ 中的区间 $I(k)$ 不是无效区间则称其为 $S(i)$ 中的有效区间。



带权区间最短路问题



- 区间 $I(2)$ 是 $S(4)$ 的无效区间， $I(3)$ 是 $S(3)$ 的有效区间， $I(5)$ 是 $S(5)$ 的无效区间， $I(9)$ 是 $S(10)$ 的无效区间， $I(10)$ 是 $S(10)$ 的有效区间



带权区间最短路问题

- 性质1:** 区间 $I(k)$ 是 $S(i)$ 中的有效区间, 则对任意 $k \leq j \leq i$, 区间 $I(k)$ 是 $S(j)$ 中的有效区间。另一方面, 若区间 $I(k)$ 是 $S(i)$ 中的无效区间, 则对任意 $j > i$, 区间 $I(k)$ 是 $S(j)$ 中的无效区间。
- 性质2:** 集合 $S(i)$ 中所有有效区间的并覆盖从 $a(1)$ 到 $b(j)$ 的线段, 其中 $b(j)$ 是 $S(i)$ 的最右有效区间的右端点。
- 性质3:** 区间 $I(i)$ 是集合 $S(i)$ 中的有效区间当且仅当在 $S(i)$ 中有一条从 $I(1)$ 到 $I(i)$ 的路。
- 性质4:** 当 $i > k$ 且 $\text{dist}(i, i) < \text{dist}(k, i)$ 时, $I(k)$ 是 $S(i)$ 中的无效区间。



带权区间最短路问题

- 性质5:** 设 $I(j(1)), I(j(2)), \dots, I(j(k))$ 是 $S(i)$ 中的有效区间, 且 $j(1) < j(2) < \dots < j(k) \leq i$, 则 $\text{dist}(j(1), i) \leq \text{dist}(j(2), i) \leq \dots \leq \text{dist}(j(k), i)$ 。
- 性质6:** 如果区间 $I(i)$ 包含区间 $I(k)$ (因此 $i > k$) , 且 $\text{dist}(i, i) < \text{dist}(k, i)$, 则 $I(k)$ 是 $S(i)$ 中的无效区间。
- 性质7:** 当 $i > k$ 且 $\text{dist}(i, i) < \text{dist}(k, i-1)$ 时, $I(k)$ 是 $S(i)$ 中的无效区间。
- 性质8:** 如果区间 $I(k) (k > 1)$ 不包含 $S(k-1)$ 中任一有效区间 $I(j)$ 的右端点 $b(j)$, 则对任意 $i \geq k$, $I(k)$ 是 $S(i)$ 中的无效区间。



带权区间图的最短路算法

算法shortestIntervalPaths

步骤1: $\text{dist}(1,1) \leftarrow w(1);$

步骤2:

for ($i=2; i \leq n; i++$) {

(2.1):

$j = \min\{k \mid a(i) < b(k); 1 \leq k < i\};$

if (j 不存在) $\text{dist}(i,i) \leftarrow +\infty;$

else $\text{dist}(i,i) \leftarrow \text{dist}(j,i-1) + w(i);$

(2.2):

for ($k < i$) {

if ($\text{dist}(i,i) < \text{dist}(k,i-1)$) $\text{dist}(k,i) \leftarrow +\infty;$

else $\text{dist}(k,i) \leftarrow \text{dist}(k,i-1);$

}

}

步骤3:

for ($i=2; i \leq n; i++$) {

if ($\text{dist}(i,n) = +\infty$) {

$j = \min\{k \mid (\text{dist}(k,n) < +\infty) \&\& (a(i) < b(k))\};$

$\text{dist}(i,n) = \text{dist}(j,n) + w(i);$

}

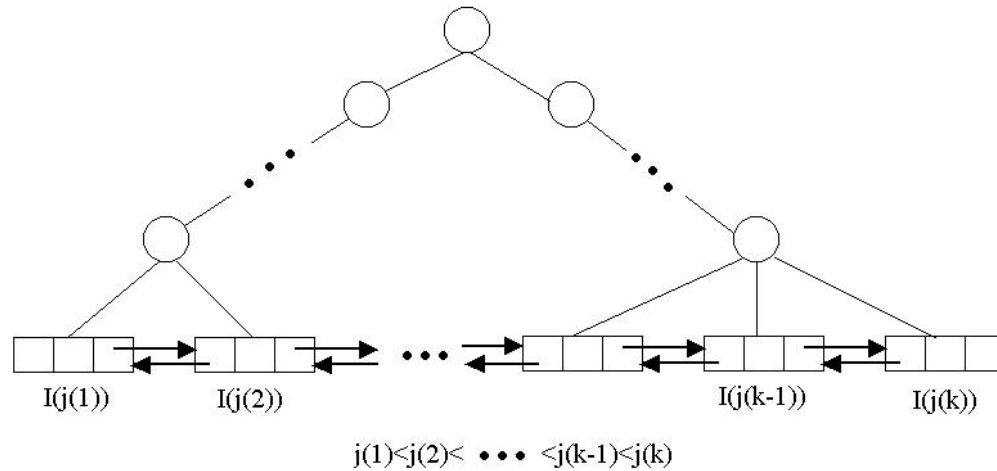
}

上述算法的关键是有效地实现步骤(2.1)和(2.2)



实现方案1

用一棵平衡搜索树（2-3树）存储当前区间集 $S(i)$ 中的有效区间。以区间的右端点的值为序。如图所示。



(2.1)的实现对应于平衡搜索树从根到叶的一条路径上的搜索，在最坏情况下需要时间 $O(\log n)$ 。(2.2)的实现对应于反复检查并删除平衡搜索树中最右叶结点的前驱结点。在最坏情况下，每删除一个结点需要时间 $O(\log n)$ 。

综上，算法**shortestIntervalPaths**用平衡搜索树的实现方案，在最坏情况下的计算时间复杂性为 $O(n \log n)$ 。



实现方案2

采用并查集结构。用整数 k 表示区间 $I(k)$, $1 \leq k \leq n$ 。初始时每个元素 k 构成一个单元素集，即集合 k 是 $\{k\}$, $1 \leq k \leq n$ 。

- (1) 每个当前有效区间 $I(k)$ 在集合 k 中。
- (2) 对每个集合 $S(i)$ ，设 $L(S(i)) = \{I(k) | I(k) \text{ 是 } S(i) \text{ 的无效区间, 且 } I(k) \text{ 与 } S(i) \text{ 的任一有效区间均不相交}\}$ ， $L(S(i))$ 中所有区间均位于 $S(i)$ 的所有有效区间并的右侧。
- (3) 用一个栈 AS 存放当前有效区间 $I(i(1))$, $I(i(2))$, ..., $I(i(k))$ 。 $I(i(k))$ 是栈顶元素。该栈称为当前有效区间栈。
- (4) 对于 $1 \leq k \leq n$ ，记 $\text{prev}(I(k)) = \min\{j | a(k) < b(j)\}$ 。对给定的区间序列做一次线性扫描确定 $\text{prev}(I(k))$ 的值。
- (5) 对于当前区间集 $S(i)$ ，用一维数组 dist 记录 $\text{dist}(j, i)$ 的值。
- (6) 用 $\text{dist}[k] = -1$ 标记区间 $I(k)$ 为无效区间。



优化搜索策略



最短加法链问题

给定一个正整数和一个实数，如何用最少的乘法次数计算出 x^n 。例如，可以用6次乘法逐步计算 x^{23} 如下：

$$x, x^2, x^3, x^5, x^{10}, x^{20}, x^{23}$$

可以证明计算最少需要6次乘法。计算的幂序列中各幂次1, 2, 3, 5, 10, 20, 23组成了一个关于整数23的加法链。在一般情况下，计算 x^n 的幂序列中各幂次组成正整数的一个加法链

$$1 = a_0 < a_1 < a_2 < \cdots < a_r = n$$

$$a_i = a_j + a_k, k \leq j < i, i = 1, 2, \cdots, r$$

上述最优求幂问题相应于正整数的最短加法链问题，即求 n 的一个加法链使其长度达到最小。正整数的最短加法链长度记为 $l(n)$ 。



```
private static void backtrack(int step)
```

```
int i,j,k;
```

```
{ if (step<best) 更新最优值
```

```
return;
```

}

```
for (i=step;i>=1;i--)
```

```
if (2*a[i]>a[step])
```

```
for (j=i;j>=1;j--){
```

```
k=a[i]+a[j];
```

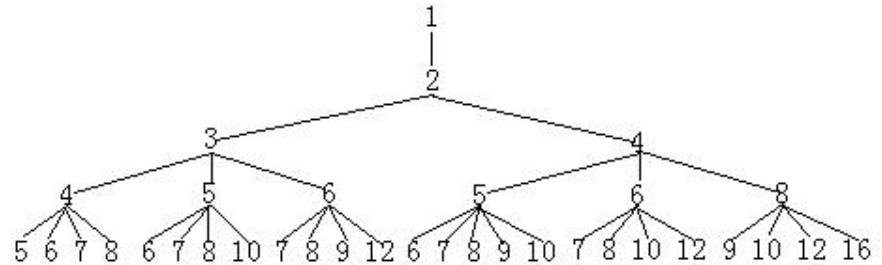
```
a[step+1]=k;
```

```
if ((k>a[step])&&(k<=n))
```

```
backtrack(step+1);
```

}

}



2026-4-26



迭代搜索法

- 深度优先搜索: 算法所搜索到的第一个加法链不一定是最短加法链。
- 广度优先搜索: 算法找到的第一个加法链就是最短加法链, 但这种方法的空间开销太大。
- **迭代搜索算法:** 既能保证算法找到的第一个加法链就是最短加法链, 又不需要太大的空间开销。其基本思想是控制回溯法的搜索深度 d , 从 $d=1$ 开始搜索, 每次搜索后使 d 增1, 加深搜索深度, 直到找到一条加法链为止。



迭代搜索法

```
private static void iterativeDeepening()  
{// 逐步深化的迭代搜索算法  
    best=n+1;  
    found=false;  
    lb=2; // 初始迭代搜索深度  
    while (!found){  
        a[1]=1;  
        backtrack(1);  
        lb++; // 加深搜索深度  
    }  
}
```

对于正整数，记 $\lambda(n)=\lfloor \log n \rfloor$ ， $v(n)=n$ 的2进制表示中1的个数。迄今为止所知道的 $l(n)$ 的最好下界是 $l(n) \geq lb(n) = \lambda(n) + \lceil \log v(n) \rceil$ 。利用这个下界，可以从深度 $lb(n)$ 开始搜索，大大加快了算法的搜索进程。



剪枝函数

- 设 a_i 和 a_j 是加法链中的两个元素，且 $a_i > 2^m a_j$ 。由于加倍是加法链中元素增大的最快的方式，即 $a_i \leq 2a_{i-1}$ ，所以从 a_j 到 a_i 至少需要 $m+1$ 步。如果预期在状态空间树 T 的第 d 层找到关于 n 的一条加法链，则以状态空间树第 i 层结点 a_i 为根的子树中，可在第 d 层找到一条加法链的必要条件是 $2^{d-i}a_i \geq n$ 。
- 当 n 是奇数时，加法链最后一个元素 $a_r = a_j + a_k$ ， $k \leq j$ ，是奇数，于是 $k < j$ (否则 a_r 为偶数)，由 r 最小可知， $j = r-1$ 。因此， $a_r = a_{r-1} + a_k$ ， $k < r-1$ 。如果在第 d 层找到最短加法链，即 $r = d$ ，则 $a_{d-1} + a_{d-2} \geq n$ ，由于 $a_{d-1} \leq 2a_{d-2}$ ，所以 $3a_{d-2} \geq n$ ，由 $a_{d-2} \leq 2a_{d-3}$ 知 $6a_{d-3} \geq n$ ，一般的，有 $3 \cdot 2^{d-(i+2)} a_i \geq n$ ，当 $3 \cdot 2^{d-(i+2)} a_i < n$ 时，状态空间树种以 a_i 为根节点的子树不可能在第 d 层之前找到最短加法链，因此可以剪去。
$$\begin{cases} \log(n / 3a_i) + i + 2 > d & 0 \leq i \leq d - 2 \\ \log(n / a_i) + i > d & d - 1 \leq i \leq d \end{cases}$$



剪枝函数

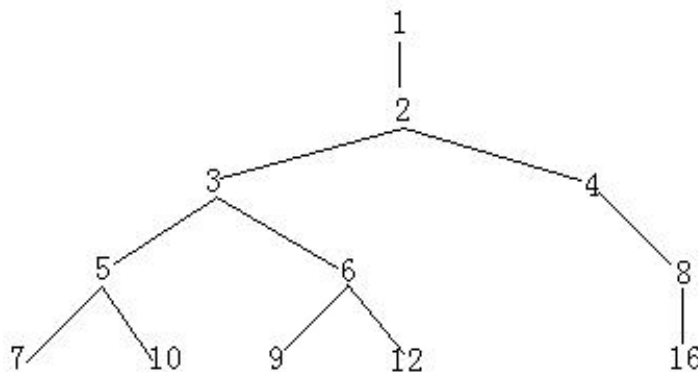
- 对于 n 为偶数时，当 n 是2的幂，唯一最短加法链是 $1, 2, 4, 8, \dots, 2^m$. 当 n 不是2的幂，可将 n 表示为 $n = 2^t(2k+1), k \geq 1$
- 设在求正整数 n 的最短加法链的逐步深化迭代搜索算法中，当前搜索深度为 d 。且正整数可表示为 $n = 2^t(2k+1), k > 0$ ，则在状态空间树的第 i 层结点 a_i 处的一个剪枝条件是

$$\begin{cases} \log(n / 3a_i) + i + 2 > d & 0 \leq i \leq d - t - 2 \\ \log(n / a_i) + i > d & d - t - 1 \leq i \leq d \end{cases}$$



最短加法链长的上界

与加法链问题密切相关的幂树给出了 $l(n)$ 的更精确的上界。



假设已定义了幂树 T 的第 k 层结点，则 T 的第 $k+1$ 层结点可定义如下。依从左到右顺序取第 k 层结点 a_k ，定义其按从左到右顺序排列的儿子结点为 $a_k + a_j$ ， $0 \leq j \leq k$ 。其中 $a_0, a_1, \dots, a_k$ ，是从 T 的根到结点 a_k 的路径。且 $a_k + a_j$ 在 T 中未出现过。

含正整数 n 的部分幂树 T 容易在线性时间内用迭代搜索方式构造出来。



优化算法

综合前面的讨论，对构造最短加法链的标准回溯法作如下改进。

- (1) 采用逐步深化迭代搜索策略；
- (2) 利用 $l(n)$ 的下界 $lb(n)$ 对迭代深度作精确估计；
- (3) 采用剪枝函数对问题的状态空间树进行剪枝搜索，加速搜索进程；
- (4) 用幂树构造 $l(n)$ 的精确上界 $ub(n)$ 。

当 $lb(n)=ub(n)$ 时，幂树给出的加法链已是最短加法链。

当 $lb(n)<ub(n)$ 时，用改进后的逐步深化迭代搜索算法，从深度 $d=lb(n)$ 开始搜索。